

AES-128 VLSI PROJECT — BRAIN / ROADMAP

Resource baseline: AES128_ALL_VERILOG_CODE(2).txt

Goal: turn the supplied AES-128 RTL implementation into a complete, demonstrable VLSI project with verified functionality and measurable hardware results.

1. Project in one picture

The project is a 128-bit AES cryptographic hardware engine. It accepts a 128-bit data block and 128-bit key, supports encryption/decryption selection, and produces a 128-bit output with a done indication. The supplied top-level module is `aes128_top`, which selects the encryption or decryption datapath using `mode`.

Input	clk, rst, start, mode, data_in[127:0], key[127:0]
Top	aes128_top
Encryption	aes128_encrypt
Decryption	aes128_decrypt
Transforms	SubBytes / ShiftRows / MixColumns + inverse blocks
Key	key_expand + AES S-box + Rcon
Verification	aes128_tb with known AES test vectors
Final VLSI stage	Simulation → synthesis → timing → area → power → implementation

2. Module map — build in this order

`aes_sbox` — 8-bit AES substitution lookup. Treat as a core combinational block.

`aes_inv_sbox` — 8-bit inverse substitution lookup for decryption.

`sub_bytes / inv_sub_bytes` — Apply the S-box independently to the 16 bytes of the 128-bit state.

`shift_rows / inv_shift_rows` — Rearrange the 16 state bytes according to AES row shifts.

`mix_columns / inv_mix_columns` — Finite-field column transformations.

`key_expand` — Generate the next 128-bit round key from the current key and round number.

`aes128_encrypt` — Clocked 10-round encryption controller/datapath.

`aes128_decrypt` — Round-key generation/storage and inverse decryption controller.

`aes128_top` — External interface and mode selection.

`aes128_tb` — Functional verification using known plaintext/key/ciphertext vectors.

3. Exact workflow to follow

Step 1 — Freeze the baseline: Keep the uploaded resource unchanged as the reference implementation. Create a separate project copy for modifications.

Step 2 — Create RTL project: Add every .v module, set `aes128_top` as synthesis top, and keep `aes128_tb` as simulation top.

Step 3 — Compile: Run RTL elaboration/compile. Fix only actual syntax/elaboration issues; do not change the AES algorithm casually.

Step 4 — Simulate encryption: Run the supplied standard test vector and confirm ciphertext = 69c4e0d86a7b0430d8cdb78070b4c55a.

Step 5 — Simulate decryption: Feed that ciphertext back with mode=decrypt and confirm plaintext = 00112233445566778899aabbccddeeff.

Step 6 — Inspect waveforms: Capture clk, rst, start, mode, round, busy/done, state and data_out where available.

Step 7 — Synthesize: Run synthesis for FPGA first if using Vivado, or an ASIC synthesis flow if available. Record LUT/FF/BRAM/DSP, area, timing and power as applicable.

Step 8 — Identify critical paths: Check timing reports and locate the dominant logic, likely involving S-box/finite-field logic or key expansion depending on the synthesized architecture.

Step 9 — Optimize: Only after a working baseline: consider pipelining, resource sharing, S-box implementation changes, clock enable/clock gating, or round architecture changes.

Step 10 — Re-verify: After every optimization, rerun functional simulation and compare outputs against the same reference vectors.

Step 11 — Final evidence: Collect RTL hierarchy, simulation waveform, PASS output, synthesis report, timing summary, utilization/area, and power estimate.

4. VLSI mindset

Do not stop at “the Verilog works.” A VLSI project should answer: How much hardware does it use? How fast does it run? How much power does it consume? What is the critical path? Which block dominates area/timing? Can the architecture be improved?

Baseline → Measure → Optimize → Measure again is the central loop.

5. Expected final deliverables

- Complete Verilog RTL source
- Simulation/testbench and PASS results
- RTL hierarchy/block diagram
- Waveform screenshot
- Synthesis result
- Timing / maximum-frequency result
- Area or FPGA utilization result
- Power estimate if the chosen tool supports it
- Comparison table before vs after optimization
- Final report + presentation + GitHub repository

6. What NOT to claim

The supplied file demonstrates RTL-level AES hardware and functional verification. It does not by itself prove ASIC fabrication, silicon tape-out, physical layout, or measured silicon power. Those claims require corresponding implementation/evidence.

7. Final project story

“I implemented an AES-128 cryptographic engine in Verilog HDL at RTL level, verified encryption and decryption against standard test vectors, then synthesized and analyzed the design for hardware metrics such as timing, area/utilization and power. I used the measurements to identify optimization opportunities and re-verified the optimized architecture.”